LIST OF FIGURES

FIGURE NO.	NAME OF THE FIGURE	PAGE NO.
3.1	SYSTEM ARCHITECTURE	9
4.1	AWS LEARNERS LAB STATUS ACTIVE	13
4.2	AWS CONSOLE HOME	13
4.3	ACTIVITY DIAGRAM FOR DATA ENGINEERING MODULE	14
5.1	AMAZON S3 BUCKET CREATED FOR CROWD SAFETY DATA,RESULTS.	17
5.2	AMAZON ATHENA QUERY RESULTS	18
5.3	AMAZON API GATEWAY API DETAILS	18
5.4	ACTIVITY DIAGRAM FOR CLOUD ENGINEERING	19
6.1	AWS LAMBDAT FOR CROWD-LIVE DATA	22
6.2	LAMBDA FUNCTION CODE CROWD ANALYTICS	23
6.3	AWS LAMBDA FUNCTION DASHBOARD	23
6.4	AWS LAMBDA TEST EVENT CONFIGURATION	23
6.5	MONITORING OF EACH GATE	24
6.6	CROWD SAFETY ASSISTANCE	24
6.7	ACTIVITY DIAGRAM FOR AI ENGINEERING MODULE	25
7.1	AMAZON S3 SECURITY CONFIGURATION	28
7.2	AWS CLOUDWATCH SERVICE	28
7.3	VISUALIZATION OF ANOMALY DETECTION	29
7.4	ACTIVITY DIAGRAM FOR SECURITY ENGINEERING	30

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
AWS	Amazon Web Services
CORS	Cross-Origin Resource Sharing
ETL	Extract, Transform, and Load
EC2	Elastic Compute Cloud
ETL	Extract, Transform, Load
HTTP	Hypertext Transfer Protocol
IAM	Identity and Access Management
IaaS	Infrastructure as a Service
JSON	JavaScript Object Notation
LLM	Large Language Model
MaaS	Model-as-a-Service
ML	Machine Learning
NLP	Natural Language Processing
PaaS	Platform as a Service
SQL	Structured Query Language
REST	Representational State Transfer
SaaS	Software as a Service

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	LIST OF FIGURES	**iii**
	LIST OF ABBREVIATIONS	**iv**
1.	**INTRODUCTION**	**1**
	1.1 DATA ENGINEERING MODULE	1
	1.2 AI ENGINEERING MODULE	2
	1.3 CLOUD ENGINEERING MODULE	3
	1.4 SECURITY ENGINEERING MODULE	4
2.	**PROBLEM STATEMENT**	**6**
3.	**SYSTEM ARCHITECTURE**	**8**
	3.1 INTRODUCTION	8
	3.2 CLIENT LAYER & NETWORK SECURITY	9
	3.3 APPLICATION LAYER	10
	3.4 DATA LAYER AND BENEFITS	10
4.	**MODULE 1 : DATA ENGINEERING**	**12**
	4.1 AIM	12
	4.2 PROCEDURE	12
	4.3 SCREENSHOTS OF DATA ENGINEERING	13
	4.4 ACTIVITY DIAGRAM	14
	4.5 RESULTS	15
5.	**MODULE 2 : CLOUD ENGINEERING**	**16**
	5.1 AIM	16
	5.2 PROCEDURE	16

	5.3	SCREENSHOTS OF CLOUD ENGINEERING	17
	5.4	ACTIVITY DIAGRAM	19
	5.5	RESULTS	20
6.	**MODULE 3 : AI ENGINEERING**		**21**
	6.1	AIM	21
	6.2	PROCEDURE	21
	6.3	SCREENSHOTS OF AI ENGINEERING	22
	6.4	ACTIVITY DIAGRAM	25
	6.5	RESULTS	26
7.	**MODULE 4 : SECURITY ENGINEERING**		**27**
	7.1	AIM	27
	7.2	PROCEDURE	27
	7.3	SCREENSHOTS OF SECURITY ENGINEERING	28
	7.4	ACTIVITY DIAGRAM	30
	7.5	RESULTS	31
8.	**CONCLUSION**		**32**
	REFERENCES		**33**

CHAPTER 1
INTRODUCTION

1.1 DATA ENGINEERING

1.1.1 What is Data Engineering?

Data Engineering is a specialized discipline within the broader field of computer science and information technology that focuses on the design, construction, and maintenance of systems and pipelines responsible for collecting, storing, transforming, and delivering large volumes of data in a reliable and efficient manner. It serves as the foundational layer of any data-driven application, ensuring that raw data generated from diverse sources is systematically processed into a structured and meaningful format that can be consumed by analytics engines, machine learning models, and decision-making systems [16] [13].

1.1.2 Types of data models

A data model is a conceptual framework that defines the structure, organization, and relationships of data within a system, serving as the blueprint for how data is stored, accessed, and managed across different layers of an application architecture [13]. The three primary types of data models conceptual, logical, and physical represent progressively detailed levels of abstraction, where the conceptual model defines high-level business entities and their relationships, the logical model specifies attributes, data types, and constraints without reference to any specific technology, and the physical model describes the actual implementation of data storage including table structures, indexing strategies, and partitioning schemes within a chosen platform [13][16].

1.1.3 Data pipeline models

A data pipeline model defines the structured flow of data from its origin source through transformation and processing stages to final storage and consumption, ensuring data moves reliably and efficiently across all system components [13]. The three primary pipeline models include batch processing which handles large volumes of data at scheduled intervals for historical analysis stream processing which continuously processes real-time data with low latency and hybrid models that combine approaches to balance accuracy and responsiveness [16].

In AWS cloud environments, pipeline models are implemented using Amazon S3 for raw data storage, AWS Glue for serverless ETL transformation and cataloging, Amazon Athena for SQL-based querying, and AWS Lambda for event-driven real-time processing [1][2]. In the Campus Crowd Safety System, raw crowd JSON records ingested into S3 are transformed through the Glue crawler, historically analyzed through Athena SQL queries, and consumed by Lambda functions that compute real-time Z-score anomaly detections and serve live gate occupancy data to the React dashboard through API Gateway [4][14].

.

1.2 AI ENGINEERING

1.2.1 What is AI Engineering?

AI Engineering is a multidisciplinary field that combines principles of software engineering, data science, and machine learning to design, build, deploy, and maintain intelligent systems capable of performing complex tasks that traditionally require human intelligence including pattern recognition, natural language understanding, anomaly detection, and decision support [15]. It bridges the gap between research-level AI models and production-ready applications by establishing scalable infrastructure, automated pipelines, and robust deployment frameworks that ensure AI systems operate reliably, securely, and efficiently at scale [16].

1.2.2 Types of Machine Learning Models

Machine learning models are computational systems that learn patterns and relationships from data to make predictions, classifications, or decisions without being explicitly programmed for every possible scenario, and they are broadly categorized into three primary types based on the nature of their training process and the availability of labeled data. Supervised learning models are trained on labeled datasets where both input features and expected output values are provided, enabling the model to learn the mapping between inputs and outputs for tasks such as classification — assigning inputs to predefined categories and regression predicting continuous numerical values — making them suitable for crowd density classification into LOW, MEDIUM, HIGH, and CRITICAL levels [16] [13]

1.2.3 AWS Services for AI Engineering

Amazon Web Services provides a comprehensive and fully managed ecosystem of AI engineering services that enable organizations to build, train, deploy, and monitor intelligent applications at scale without managing any underlying infrastructure, covering the complete AI lifecycle from raw data ingestion through model deployment and real-time inference [14]. Amazon SageMaker serves as the central ML platform on AWS, offering integrated tools for data preparation, feature engineering, model training, hyperparameter tuning, and hosted model endpoints that serve real-time predictions through auto-scaling REST APIs eliminating the need for manual compute provisioning across any stage of the model development lifecycle [13].

1.3 CLOUD ENGINEERING

1.3.1 What is Cloud Engineering?

Cloud Engineering is a discipline that applies software engineering principles and practices to the design, deployment, management, and optimization of applications and infrastructure hosted on cloud computing platforms, enabling organizations to build scalable, secure, and cost-efficient systems without the burden of managing physical hardware or data centers. It encompasses a broad range of technical responsibilities including cloud architecture design, infrastructure provisioning, security configuration, network management, cost optimization, and continuous monitoring all performed through managed cloud services and automation tools that replace traditional on-premises infrastructure management workflows [11] [14].

1.3.2 Types of Cloud Computing

Cloud computing is broadly categorized into distinct types based on both the deployment model defining where and how the cloud infrastructure is hosted and managed and the service model defining the level of control and responsibility shared between the cloud provider and the customer together determining the most appropriate cloud strategy for a given organizational requirement [14]. Public cloud deployment involves hosting infrastructure and services on shared, provider-managed data centers accessible over the internet, offering maximum scalability, global availability, and cost efficiency through a pay-per-use pricing model making it the most widely adopted cloud deployment model for modern application development [14].

1.3.3 Cloud Deployment Models

Cloud deployment models define how cloud infrastructure is set up, managed, andaccessed by users. The four main models are Public Cloud, Private Cloud, Hybrid Cloud, and Community Cloud. In a Public Cloud, services are delivered over the internet by third-party providers such as Amazon Web Services, Microsoft Azure, and Google Cloud Platform. These platforms provide scalable resources on a pay-as-you-go basis, making them cost-effective and widely accessible. A Private Cloud, on the other hand, is dedicated to a single organization and offers greater control, security, and customization, which is especially useful for handling sensitive data and critical applications.

1.3.4 AWS Services for Cloud Engineering

A Cloud engineering on Amazon Web Services involves using a wide range of managed services to design, deploy, and maintain scalable applications in the cloud. One of the core services is **Amazon EC2 (Elastic Compute Cloud)**, which provides virtual servers for running applications with flexible configurations. For storage, **Amazon S3 (Simple Storage Service)** is widely used to store and retrieve data securely, while **Amazon RDS (Relational Database Service)** helps manage databases without worrying about underlying infrastructure. These services form the backbone of most cloud-based applications by offering reliability, scalability, and cost-efficiency [1][10].

1.4 SECURITY ENGINEERING

1.4.1 What is Security Engineering?

Security Engineering is the discipline of designing, implementing, and maintaining systems that remain secure against threats, attacks, and unauthorized access. It involves applying engineering principles to ensure the confidentiality, integrity, and availability (CIA triad) of data and systems throughout their lifecycle. Security engineering integrates various practices such as risk assessment, threat modeling, encryption, authentication, and continuous monitoring to protect both software and infrastructure from vulnerabilities.

1.4.2 Types of Cryptographic Techniques

Cryptographic techniques are used in security engineering to protect data from unauthorized access and ensure secure communication. The two main types are symmetric and asymmetric

cryptography. Symmetric encryption uses a single key for both encryption and decryption, making it fast and suitable for handling large volumes of data, while asymmetric encryption uses a pair of keys (public and private) to enhance security, especially in data transmission. Common techniques also include hashing, which converts data into a fixed-length value for integrity verification, and digital signatures, which ensure authenticity and non-repudiation.

1.4.3 Authentication Models

Authentication models define how users or systems verify their identity before accessing resources. The most common models include single-factor authentication (SFA), which relies on one credential such as a password, and multi-factor authentication (MFA), which combines two or more factors like password, OTP, or biometrics for stronger security. Other models include token-based authentication and certificate-based authentication, which are commonly used in modern applications and secure APIs.In cloud environments such as Amazon Web Services, authentication is managed through services like Identity and Access Management (IAM), which ensures that only authorized users can access specific resources based on defined roles and permissions.

1.4.4 AWS Services for Security Engineering

Security engineering in Amazon Web Services is supported by a wide range of services designed to protect applications and data. AWS Identity and Access Management (IAM) is used to control access through roles and policies, ensuring least privilege access. AWS Key Management Service (KMS) enables secure creation and management of encryption keys, while AWS Shield and AWS Web Application Firewall (WAF) provide protection against distributed denial-of-service (DDoS) attacks and web threats. Monitoring and threat detection are handled by services like Amazon CloudWatch and AWS CloudTrail, which track system activity and logs. Additionally, AWS Security Hub and Amazon GuardDuty provide centralized security management and intelligent threat detection. These services help organizations build secure, compliant, and resilient cloud systems.

CHAPTER 2
PROBLEM STATEMENT

Data engineering forms the foundation of the Campus Crowd Safety Monitoring System by enabling the collection, transformation, and storage of crowd-related data in real time. Using services such as Amazon Web Services Amazon API Gateway and AWS Lambda, the system captures live crowd metrics including entry and exit counts, zone density levels, gate-wise occupancy, and timestamp-based activity. This data is stored in Amazon S3 for scalable and durable storage, and is structured for efficient querying using Amazon Athena to generate reports and insights. AWS Glue further supports data cataloging and transformation, ensuring that raw data is converted into a usable format for analytics [2][3][4].

Cloud engineering ensures that the system is scalable, reliable, and capable of handling dynamic crowd conditions across multiple locations. By leveraging AWS Lambda and API Gateway, the backend operates in a serverless architecture that automatically scales based on incoming requests without requiring manual infrastructure management. Amazon S3 provides highly durable storage for historical crowd data, while Amazon CloudWatch continuously monitors system performance, logs, and errors in real time. This architecture ensures that large-scale events with thousands of participants can be monitored efficiently without performance degradation [1][2][3].

AI engineering introduces intelligence into the system by enabling crowd analysis and anomaly detection. Using predefined thresholds and data-driven logic, the system identifies overcrowded zones, abnormal crowd movement patterns, and peak congestion periods. Historical data can be analyzed to predict high-density time intervals and improve crowd management strategies. Additionally, an intelligent assistant powered by AI can respond to queries such as “Which gate is overcrowded?” or “What is the peak crowd time?” and provide actionable recommendations for safety management [8][9][14].

Security engineering ensures that all crowd data and system interactions are protected from unauthorized access and potential threats. With AWS Identity and Access Management (IAM), strict role-based access control is enforced for administrators and system components. Amazon S3 encryption safeguards stored data, while secure HTTPS protocols ensure safe data transmission. Monitoring tools such as Amazon CloudWatch enable logging and auditing of system activities, improving transparency, accountability, and system reliability. These measures ensure that sensitive crowd information is handled securely within the cloud environment. [1][5][6].

OBJECTIVES

- Real-Time Crowd Data Collection and Monitoring
- Cloud Infrastructure Deployment and Optimization
- AI-Powered Crowd Analysis and Anomaly Detection
- Intelligent Query Assistant for Insights
- Secure Data Storage and Access Control
- Monitoring and Performance Management
- Efficient Resource Management for Safety Optimization

CHAPTER 3

SYSTEM ARCHITECTURE

3.1 INTRODUCTION TO SYSTEM ARCHITECTURE

The system architecture of the Campus Crowd Safety Monitoring System is designed to provide a scalable, real-time, and intelligent platform for monitoring and managing crowd movement across the campus. It follows a cloud-based, serverless architecture using services from Amazon Web Services to ensure high availability, flexibility, and efficient data processing. The architecture integrates multiple components including data ingestion, storage, processing, analytics, and user interaction layers to deliver a complete end-to-end solution.

At the data ingestion layer, crowd-related data such as entry counts, exit counts, and density levels are collected in real time through APIs using Amazon API Gateway and processed by AWS Lambda functions. These serverless components enable automatic scaling based on incoming requests, eliminating the need for manual infrastructure management. The collected data is then stored in Amazon S3, which provides highly durable and secure storage for both raw and processed datasets. Data transformation and cataloging are handled using AWS Glue, allowing the system to organize data efficiently for further analysis. [1][2][4][5].

The analytics layer uses Amazon Athena to perform SQL-based queries directly on the stored data, enabling quick generation of reports and insights without the need for a dedicated database system. This allows administrators to analyze crowd patterns, identify peak times, and detect overcrowded zones effectively. Additionally, AI-based logic is integrated into the system to detect anomalies and provide intelligent recommendations for crowd control and safety management. [3][10]

Key Points:

- **Data Engineering**: Real-time data ingestion through API Gateway, processing via Lambda, storage in Amazon S3, and analytics using Athena for crowd insights.
- **Cloud Engineering**: Serverless architecture using API Gateway and Lambda ensures automatic scaling, with S3 providing durable storage and CloudWatch enabling monitoring.
- **AI Engineering**: Threshold-based anomaly detection, crowd density analysis, peak time prediction, and intelligent assistant for real-time crowd queries.

- **Security Engineering**: Secure data storage using S3 encryption, IAM-based access control, HTTPS communication, and CloudWatch logging for monitoring and audition

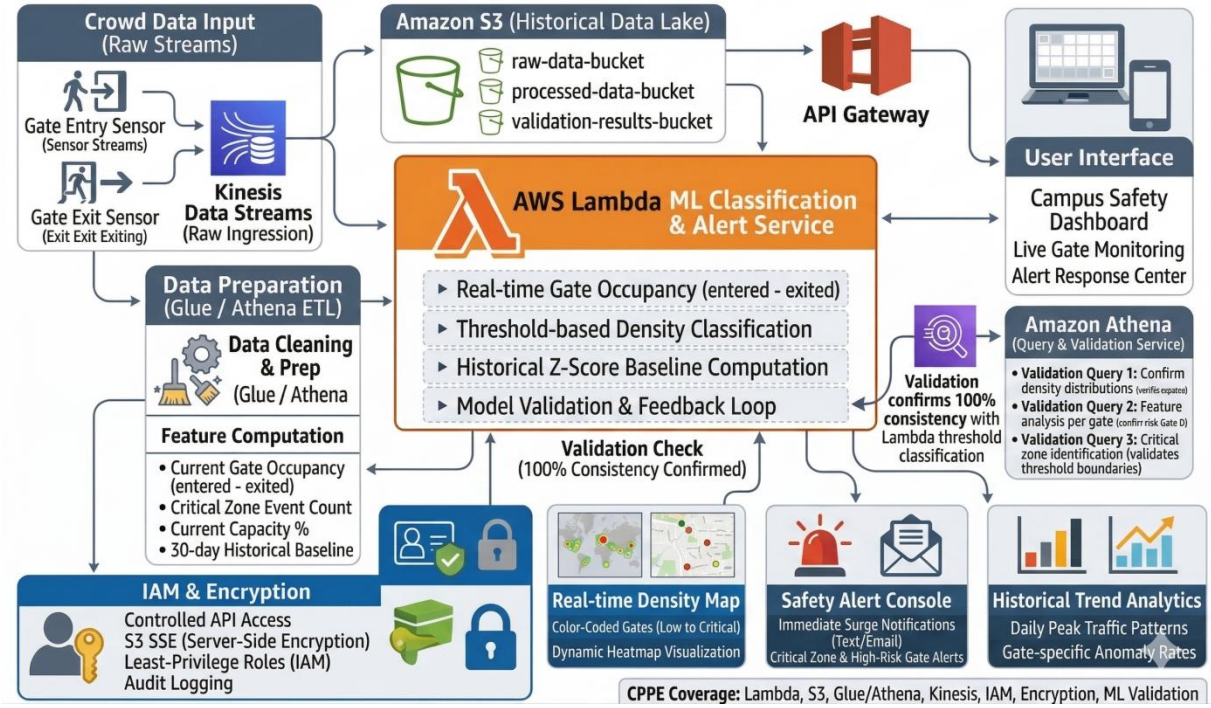


Figure 3.1 System Architect

3.2 CLIENT LAYER AND NETWORK SECURITY

The client layer represents the user interface through which administrators interact with the Campus Crowd Safety Monitoring System. It consists of a web dashboard developed using React, which communicates securely with backend services hosted on Amazon Web Services..

The client layer performs the following operations:

- Displays total crowd count across all zones.
- Shows overcrowded gates or areas.
- Displays zone-wise crowd density reports
- Shows anomaly alerts for unusual crowd behavior.
- Provides smart query input for crowd-related insights.

To maintain secure access, the following mechanisms are implemented:

- Secure login access for authorized users only.
- HTTPS communication for encrypted data transfer.
- Ccontrolled API access using IAM permissions.
- Role-based dashboard access control.

Although frontend-level controls enhance usability and basic security, backend validation and authorization mechanisms are also enforced to ensure secure data handling and trusted communication between the client and cloud services.

3.3 APPLICATION LAYER

The application layer is the core processing unit of the Campus Crowd Safety Monitoring System. It handles business logic, processes incoming requests, performs analytics, and acts as a bridge between the frontend interface and backend cloud services hosted on Amazon Web Services. This layer ensures real-time data processing, intelligent analysis, and seamless communication across the system.

The key functions of the application layer are:

- Collecting crowd data from input sources such as entry/exit points and sensors.
- Processing incoming crowd records in real time.
- Detecting overcrowded zones and high-density areas.
- Identifying unusual crowd movement or sudden spikes.
- Running analytics queries for historical and live data.
- Returning processed results to the frontend dashboard.

The backend includes the following endpoints:

- **/collect-data** – Receives crowd metrics such as counts and density.
- **/analyze** – Performs anomaly detection and crowd analysis.
- **/dashboard** – Sends summarized crowd data to the frontend.
- **/ask-query** – Handles user queries and provides intelligent responses.

This layer plays a crucial role in integrating real-time monitoring with analytical processing. It ensures efficient data flow, accurate insights, and reliable system performance for effective crowd management.

3.4 DATA LAYER AND BENEFITS

The data layer is responsible for storing and managing all crowd-related data generated by the Campus Crowd Safety Monitoring System. It handles both structured and semi-structured data using cloud storage services provided by Amazon Web Services. Amazon S3 is used to store real-time crowd logs, historical records, and analytics outputs, while Amazon Athena enables SQL-based querying directly on stored data without requiring a separate database server.

The data layer stores the following categories:

- **Crowd Logs** – Entry/exit counts, zone density, timestamps, and occupancy levels

- **Analytics Results** – Reports, summaries, anomaly detection outputs.
- **System Logs** – API requests, Lambda execution logs, alerts, and monitoring data.

Benefits of Data Layer

- Scalable storage for large volumes of crowd data.
- Fast analytics using SQL queries through Athena.
- Supports historical trend analysis and predictions.
- Reliable and secure cloud-based storage.
- Improves reporting and decision-making capabilities.
- Supports AI-based anomaly detection models.

Data Integrity

The system maintains organized storage structures, secure access permissions using IAM, encrypted data storage, and reliable backup mechanisms to ensure data safety, consistency, and high availability across the platform.

CHAPTER 4

MODULE 1 : DATA ENGINEERING

4.1 AIM :

To design and implement a secure, scalable, and efficient cloud-based data engineering pipeline using AWS services for the Campus Crowd Safety System. The module aims to collect, process, store, and analyze crowd monitoring data such as gate entry counts, exit counts, occupancy levels, zone density, and alert logs while ensuring data integrity, optimized query performance, and seamless integration with analytics and machine learning modules [14][16][17].

4.2 PROCEDURE FOR DATA ENGINEERING

Step 1: Launch and activate the AWS Academy Learner Lab environment and access the AWS Management Console [17].

Step 2: Create Amazon S3 buckets to store raw crowd data, processed analytics data, and query outputs with proper folder structure [1][14].

Step 3: Configure AWS Lambda functions to receive incoming crowd metrics such as entry count, exit count, gate occupancy, zone density levels, and timestamps [4][17].

Step 4: Create REST API endpoints using Amazon API Gateway to securely ingest real-time crowd data from frontend dashboards or simulated sources [5][19][21].

Step 5: Design structured datasets in JSON format for storing gate-wise and zone-wise crowd monitoring records inside Amazon S3 [1][2].

Step 6: Configure AWS Glue Crawler to catalog stored crowd datasets and build the data schema automatically for analytics [2][14].

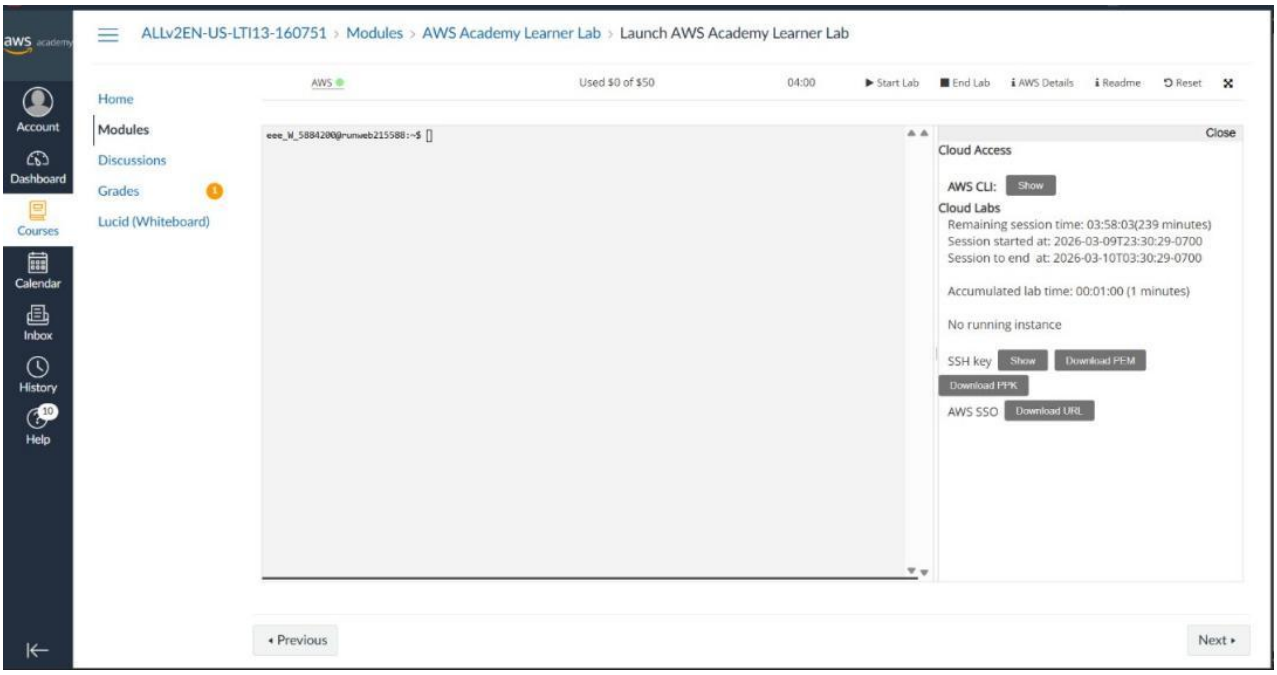
Step 7: Configure AWS Lambda functions to clean, transform, and organize incoming crowd data before storing it in Amazon S3 [4][22].

Step 8: Optimize query performance by partitioning data based on date, event, gate, or zone location for faster analytics using Amazon Athena [3][10].

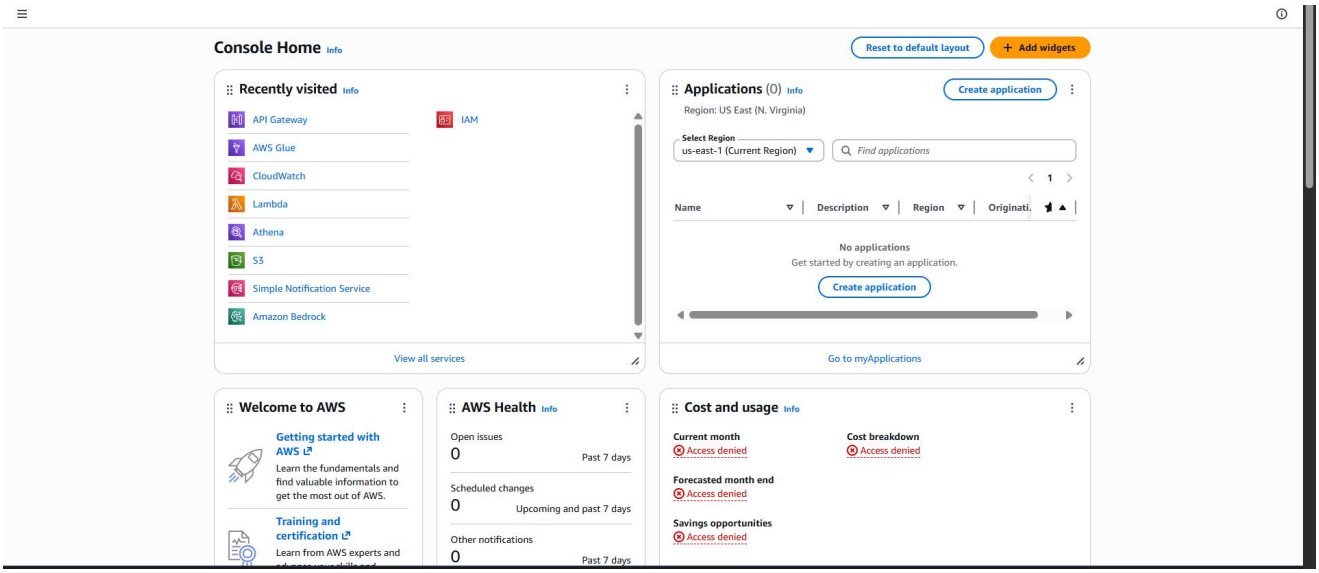
Step 9: Validate data consistency by querying stored records in Athena and verifying metrics such as overcrowded gates, peak crowd hours, and density trends [3][11][12][23].

4.3 SCREENSHOTS FOR DATA ENGINEERING

The following screenshots demonstrate the key activities performed in Data Engineering:



Screenshot 4.1 : AWS Academy Learners Lab Status Active



Screenshot 4.2 : AWS Console Home - Access to Core Services Used in the Project

4.4 ACTIVITY DIAGRAM

Activity Diagram for Data Engineering Module

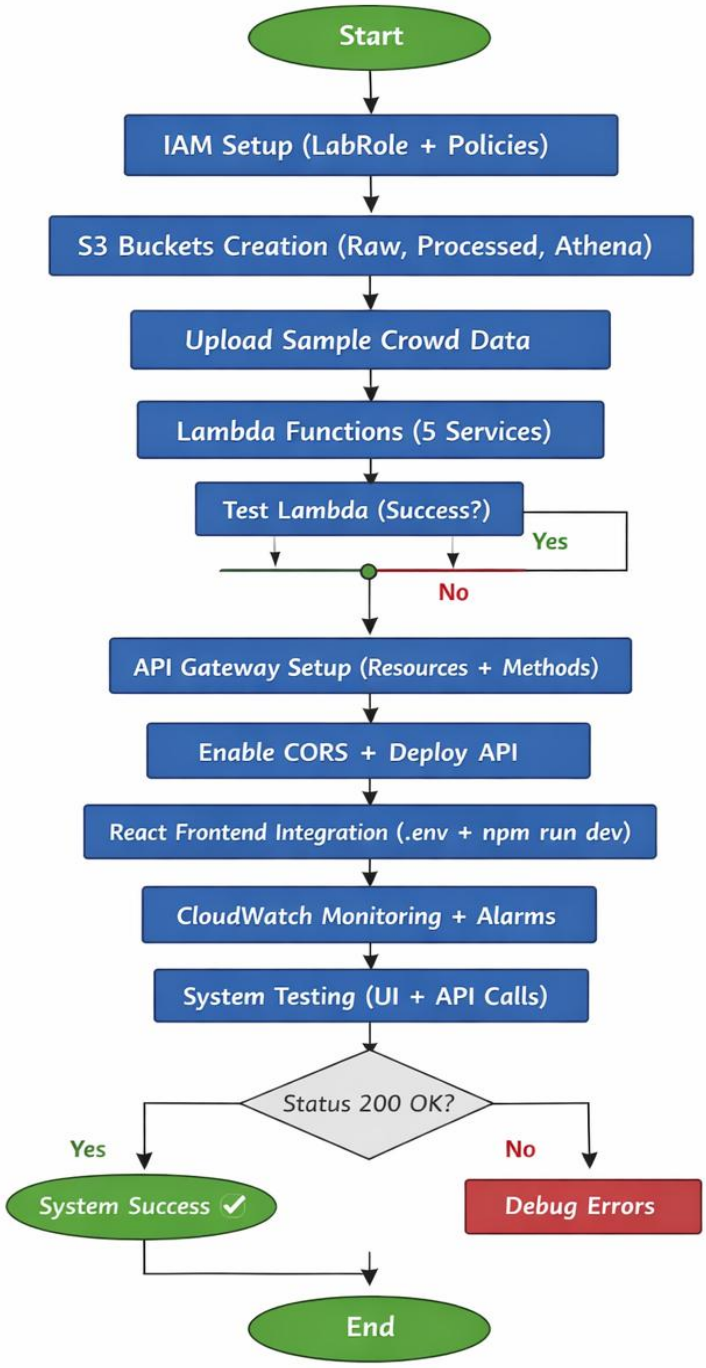


Figure 4.3 Activity Diagram for Data Engineering Module

4.5 RESULTS OF DATA ENGINEERING

The Data Engineering module was successfully completed. A scalable and well-structured cloud data pipeline was implemented using AWS services for collecting, storing, and analyzing campus crowd monitoring data. Amazon S3 was used as the centralized storage layer, while AWS Lambda, AWS Glue, and Amazon Athena were integrated for processing and analytics. [1][2][3][4]

Key Outcomes:

- Reliable storage for crowd monitoring logs, gate entry and exit records, occupancy metrics, and zone density data.
- Proper folder structure and dataset organization in Amazon S3 for efficient retrieval and future scalability.
- Seamless integration between API Gateway, Lambda, S3, Glue, and Athena for real-time data operations.
- Historical crowd data successfully prepared for congestion analysis, peak crowd prediction, and anomaly detection.
- Data integrity maintained across stored records with secure access permissions and encryption support.

This module provides a strong foundation for the subsequent Cloud Engineering, AI Engineering, and Security Engineering modules. The data platform is now ready to support real-time monitoring, intelligent analytics, and crowd safety alert features of the **Campus Crowd Safety System** [10][14][17].

CHAPTER - 5

MODULE 2: CLOUD ENGINEERING

5.1 AIM:

To deploy and configure a secure, scalable, and serverless cloud environment using AWS services for hosting the Campus Crowd Safety System, ensuring reliable backend connectivity, secure storage of crowd monitoring data, efficient processing of gate and zone occupancy metrics, and smooth communication between frontend dashboards and cloud resources through proper configurations and access control mechanisms. [1][4][6][10]

5.2 PROCEDURE FOR CLOUD ENGINEERING

Step 1: Logged into the AWS Academy Learner Lab environment and accessed the AWS Management Console to configure all cloud resources required for deployment.

Step 2: Created and configured cloud resources including Amazon S3 buckets for raw crowd data, processed data, and Athena query outputs, AWS Lambda functions for backend processing, and Amazon API Gateway for secure REST API communication.

Step 3: Configured security settings using the available **LabRole**, bucket permissions, encryption settings, and API access controls to enable secure interaction between AWS services.

Step 4: Implemented and deployed AWS Lambda backend functions for processing crowd metrics such as gate entry counts, exit counts, occupancy levels, density levels, and crowd alerts.

Step 5: Configured required backend dependencies and Python runtime libraries for Lambda functions, API communication, and analytics processing.

Step 6: Established secure communication between the React frontend and AWS cloud services using API Gateway Invoke URL, environment variables (.env), and CORS configurations.

Step 7: Created and configured Amazon S3 buckets to store crowd monitoring logs, analytics outputs, historical records, and structured JSON datasets with proper folder hierarchy.

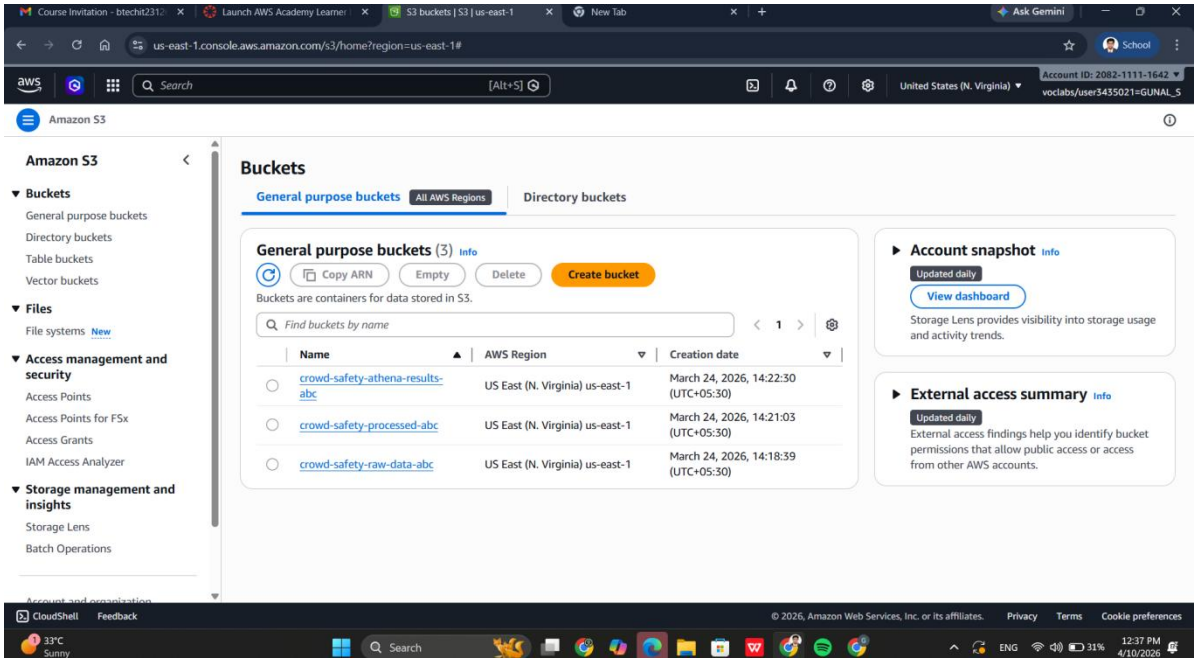
Step 8: Implemented access control using IAM LabRole policies to support secure interaction between Lambda, S3, Glue, Athena, and CloudWatch monitoring services.

Step 9: Configured Amazon Athena and executed SQL queries on stored crowd datasets to analyze overcrowded gates, peak crowd hours, and occupancy trends for real-time analytics.

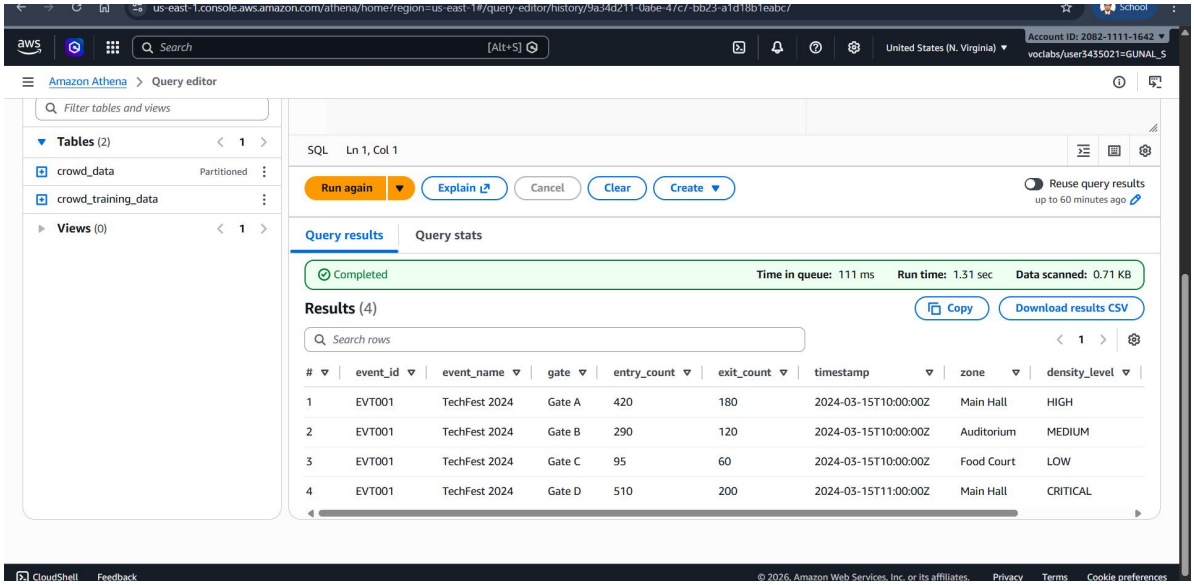
This Cloud Engineering module was successfully implemented and provides the deployment foundation for analytics, AI assistance, and security monitoring features in the **Campus Crowd Safety System**.

5.3 SCREENSHOTS FOR CLOUD ENGINEERING

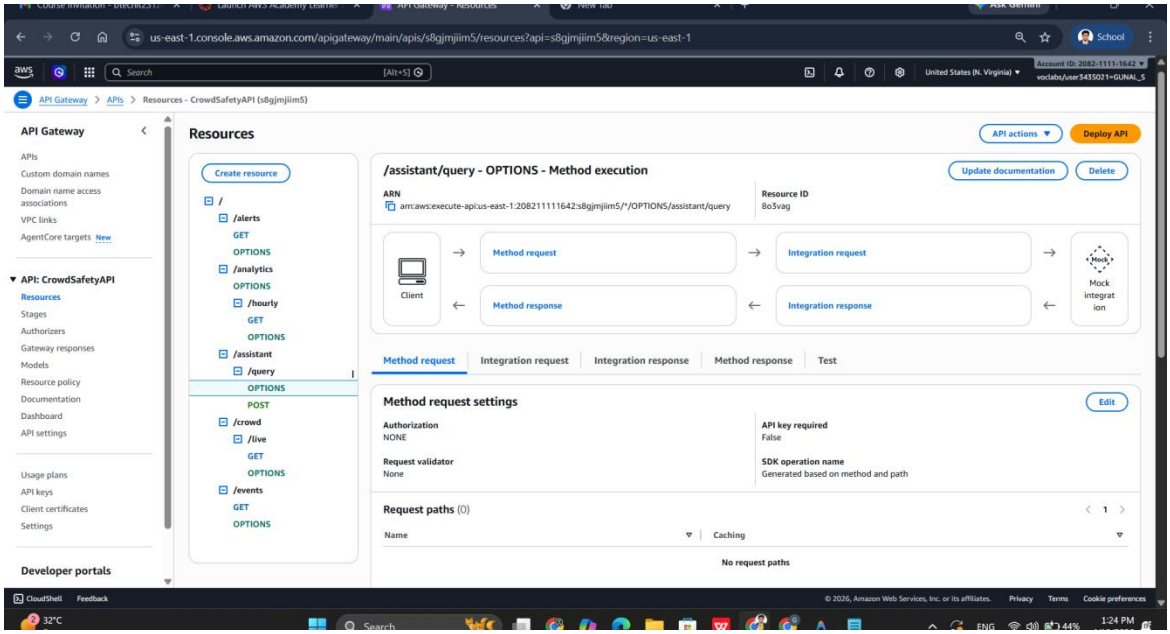
The following 4 screenshots illustrate the important stages executed in this module:



Screenshot 5.1 : Amazon S3 Bucket Created for Crowd safety data,results.



Screenshot 5.2 Amazon Athena Query Results



Screenshot 5.3 : Amazon API Gateway API Details

5.4 ACTIVITY DIAGRAM

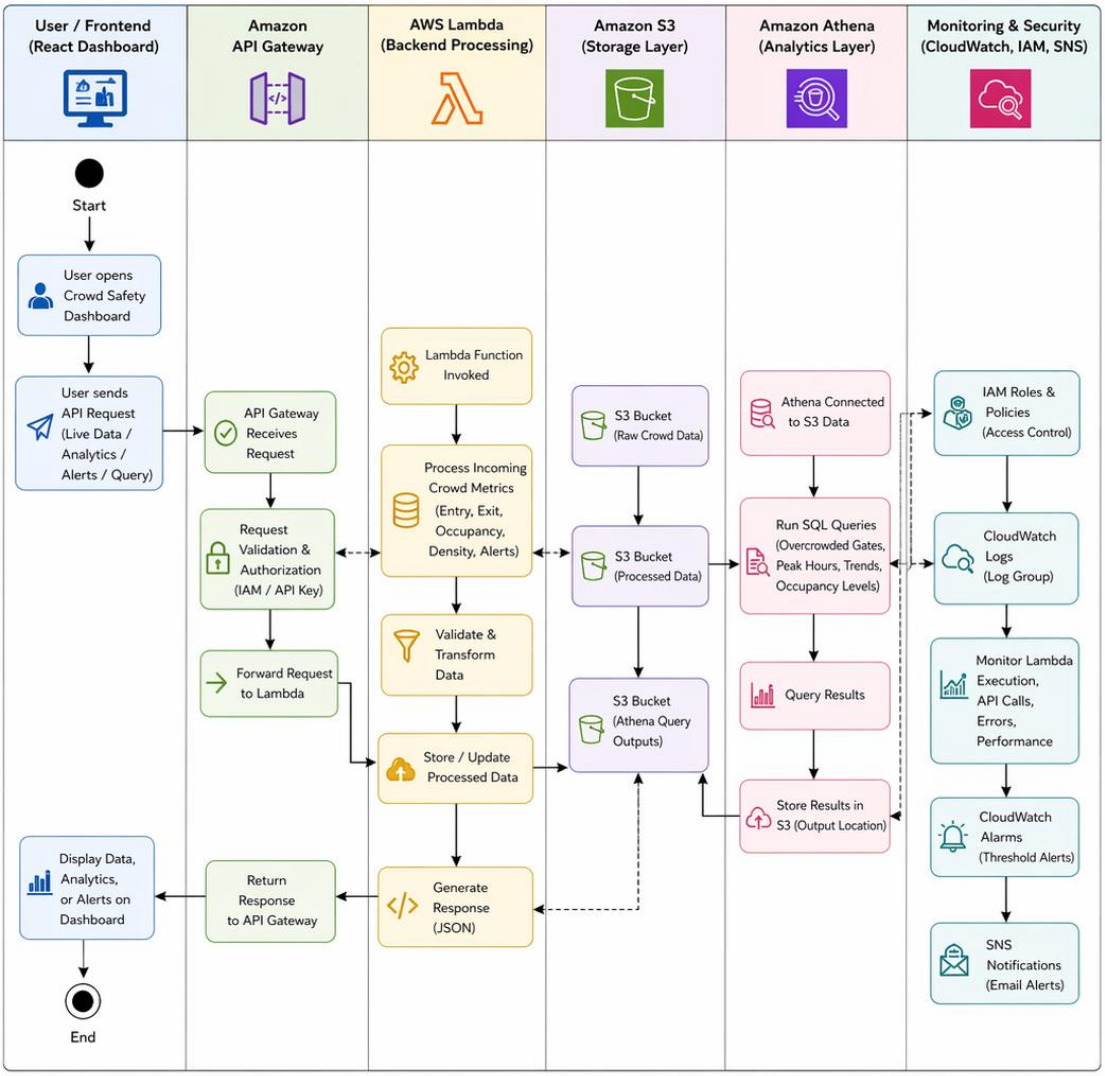


Figure 5.5 Activity Diagram for Cloud engineering

5.5 RESULT OF CLOUD ENGINEERING

The Cloud Engineering module was successfully completed. The **Campus Crowd SafetySystem** is now effectively integrated with AWS cloud services and functioning reliably.

Key Achievements:

- A secure and scalable serverless cloud infrastructure was established using AWS Lambda, Amazon API Gateway, Amazon S3, AWS Glue, and Amazon Athena. [1][3][4][5]
- The backend application was successfully connected to cloud services, enabling real-time collection, storage, and retrieval of crowd monitoring data such as gate occupancy, density levels, and alert information. [4][5]
- Cloud storage was configured using Amazon S3 to handle crowd event logs, historical records, analytics outputs, and monitoring datasets. [1][2]
- Secure communication and access control mechanisms were implemented using HTTPS configurations, API Gateway security settings, and IAM LabRole policies. [5][6][10]
- The system demonstrated stable performance with successful API responses, Lambda execution, and reliable data storage operations. [4][7]
- Amazon Athena was integrated successfully to perform SQL-based analytics on stored crowd data for peak hour analysis and overcrowding detection. [3][11][12]
- CloudWatch monitoring and alarms were configured to support log tracking, error monitoring, and system observability. [7][8]
- The cloud setup is now ready to support data engineering pipelines, AI-based crowd prediction models, and intelligent query features in subsequent modules. [15][17]

CHAPTER 6

MODULE 3 : AI ENGINEERING

6.1 AIM:

To design and integrate intelligent analytics models for detecting abnormal crowd behavior patterns in the Campus Crowd Safety System by analyzing crowd metrics such as gate occupancy, entry and exit counts, density levels, and traffic trends, ensuring accurate overcrowding detection, peak crowd prediction,bminimal false alerts, real-time performance. [11][12][13][15][16][23]

6.2 PROCEDURE FOR AI ENGINEERING

Step 1: Set up the development environment and configure required ML libraries such as scikit-learn, numpy, pandas, and joblib for implementing crowd analytics and anomaly detection models. [22][23]

Step 2: Collect and preprocess crowd monitoring data including gate entry counts, exit counts, occupancy percentages, density levels, timestamps, and zone-wise crowd records from the cloud data pipeline. [1][3][17]

Step 3: Perform feature engineering by converting raw crowd logs into meaningful inputs such as crowd density trends, occupancy thresholds, peak-hour frequency, congestion duration, and anomaly counts. [11][12][13]

Step 4: Train an intelligent analytics model (such as threshold-based anomaly detection or classification logic) to classify crowd behavior as Normal, High Density, or Critical Overcrowding. [15][16][23]

Step 5: Evaluate the model using test data to ensure prediction accuracy and reduce false alerts in overcrowding detection and crowd anomaly prediction. [23]

Step 6: Save the trained model as a serialized (.pkl) file for efficient loading and inference during runtime. [22]

Step 7: Load the trained model into the backend application and integrate it with crowd data processing logic using AWS Lambda for real-time inference. [4][16][17]

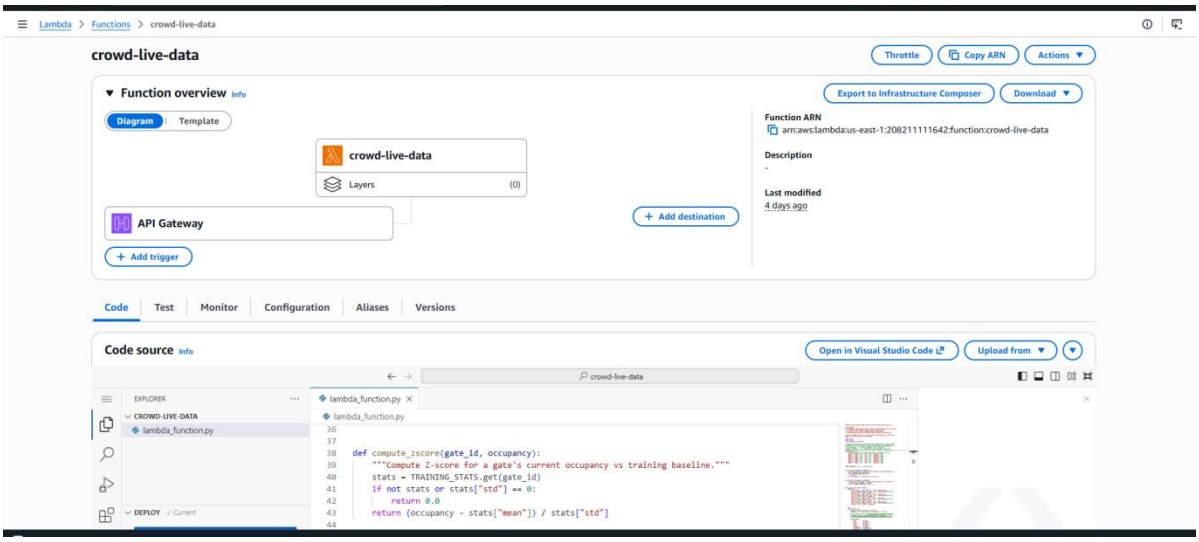
Step 8: Trigger model inference in real time by feeding current crowd metrics to generate occupancy predictions, congestion alerts, and peak crowd warnings. [15][16]

Step 9: Store model outputs along with crowd monitoring logs and analytics records for further analysis, reporting, and alert generation. [1][3][7]

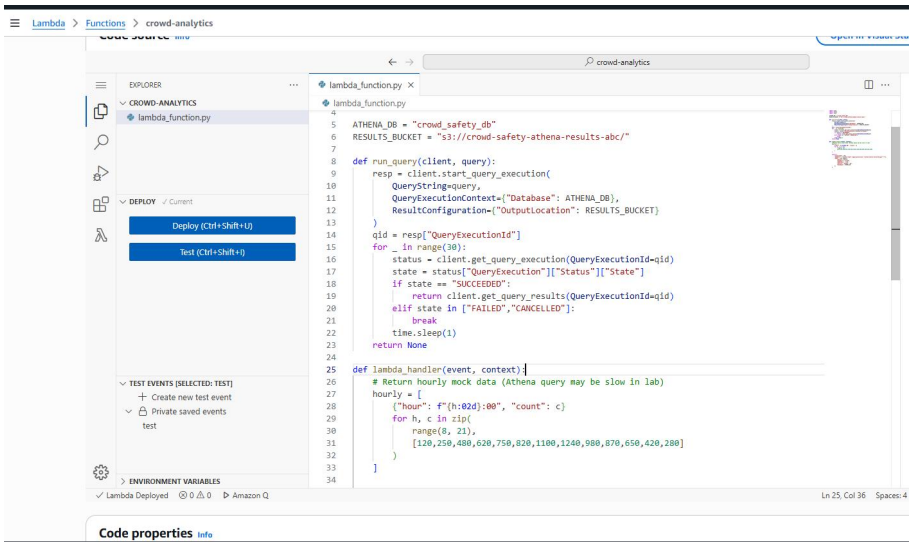
Step 10: Validate system performance by testing real-time predictions, peak crowd forecasting, low-latency responses, and reliable anomaly detection for the Campus Crowd Safety System. [11][15][23]

6.3 SCREENSHOTS OF AI ENGINEERING

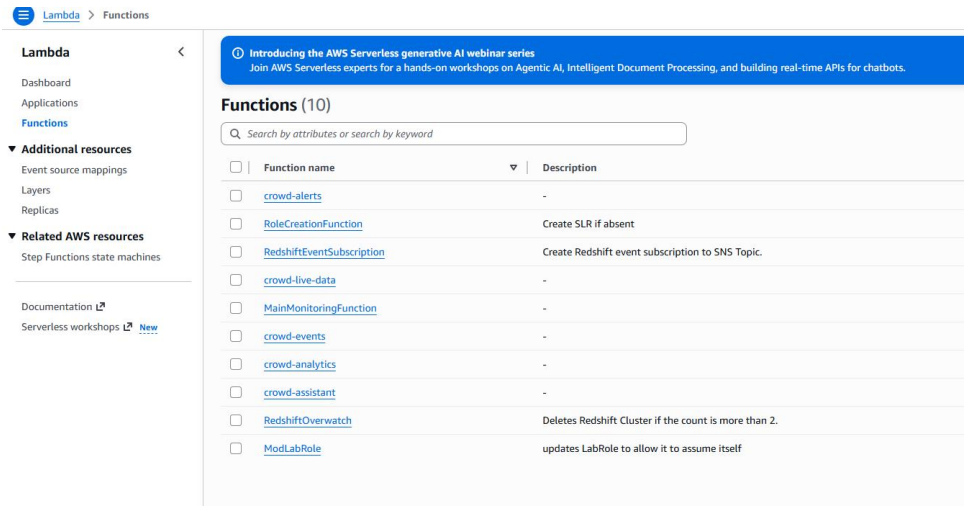
The following screenshots illustrate the important stages executed in this module:



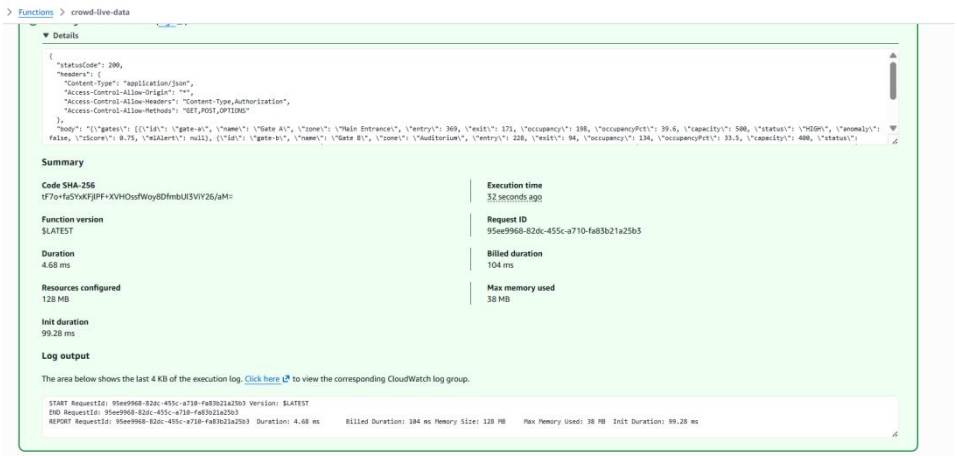
Screenshot 6.1 : AWS Lambda for crowd-live data



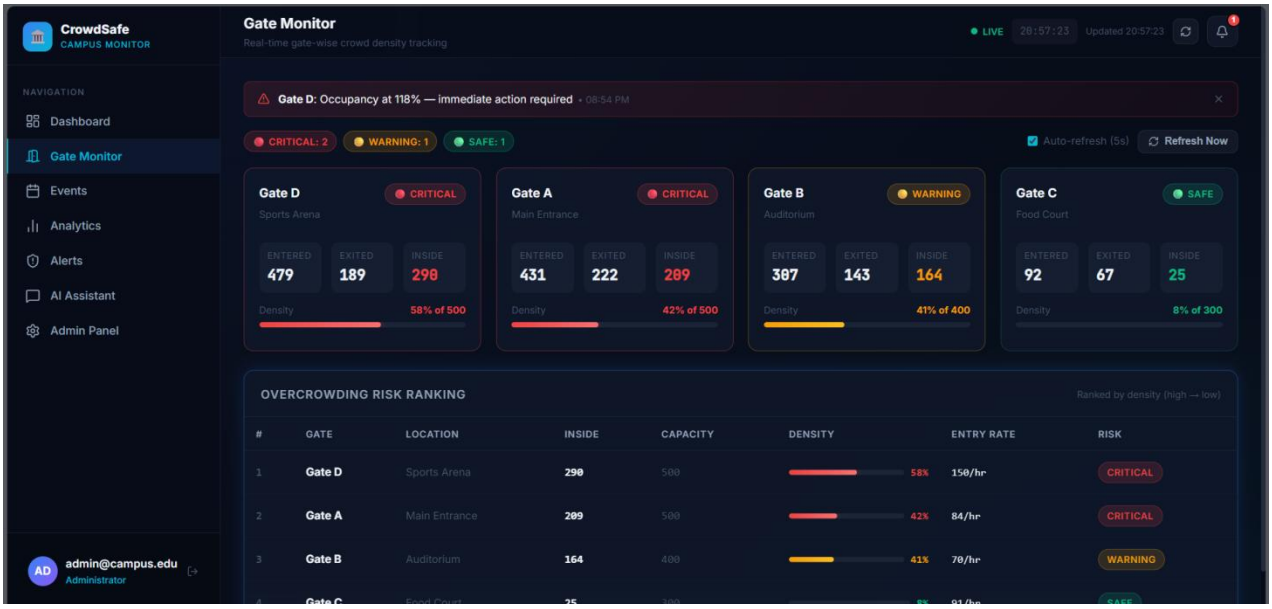
Screenshot 6.2 : Lambda Function code crowd analytics



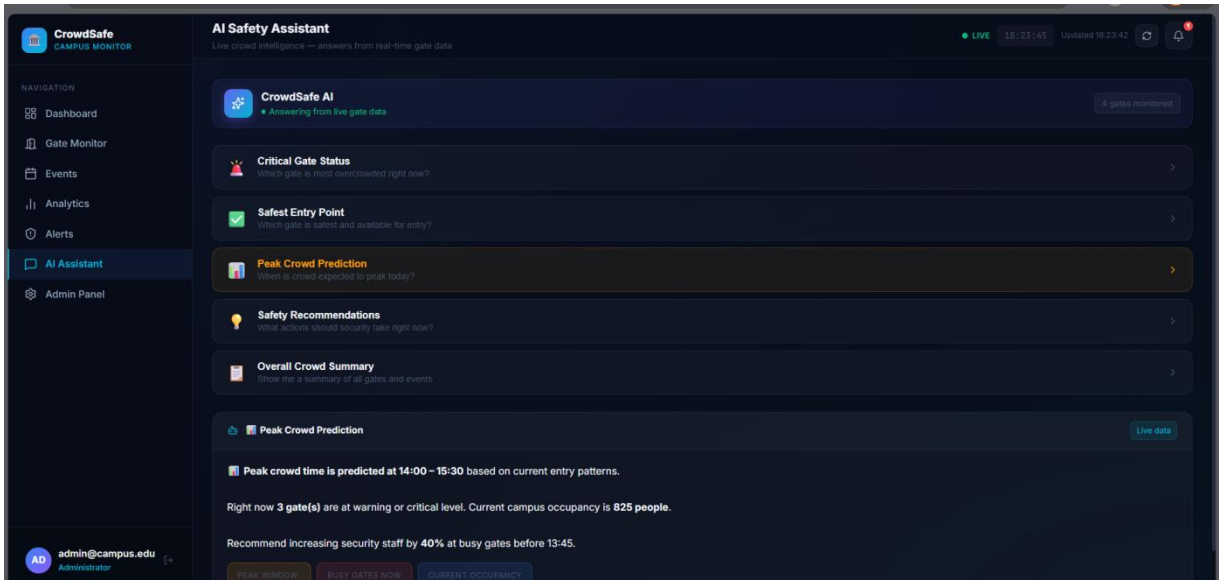
Screenshot 6.3 : AWS Lambda Functions Dashboard



Screenshot 6.4: AWS Lambda Test Event Configuration



Screenshot 6.5 : Monitoring of Each Gate



Screenshot 6.6 : crowd safety assistance

3.2 ACTIVITY DIAGRAM

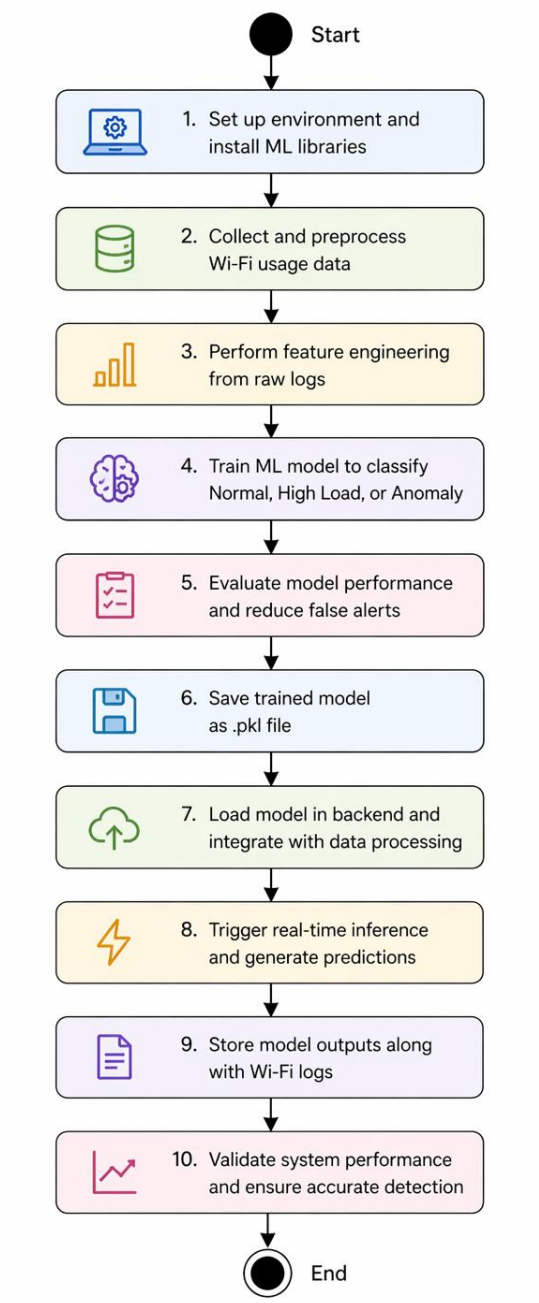


Figure 6.7 : Activity Diagram for AI Engineering Module

6.4 RESULT OF AI ENGINEERING MODULE

The AI Engineering module was successfully completed. The intelligent analytics model for detecting abnormal crowd behavior patterns was trained, integrated, and deployed into the Campus Crowd Safety System. [11][12][15][16][23]

Key Achievements:

- Intelligent analytics model (Threshold Classifier / Decision Tree / Random Forest approach) successfully implemented for detecting crowd anomalies based on occupancy and density metrics. [15][23]
- Feature-based analysis enabled using inputs such as gate entry count, exit count, occupancy levels, density spikes, and crowd movement trends. [11][12][13]
- The trained model was serialized and integrated into the backend for efficient loading and real-time inference. [22]
- The system now generates crowd status predictions such as Normal Density, High Density, or Critical Overcrowding for each monitored gate or zone. [15][16]
- Model inference is performed with low latency, ensuring real-time crowd monitoring without affecting overall system performance. [16][17]
- AI-based detection improves accuracy compared to manual crowd monitoring methods and reduces false alerts. [11][23]
- The module also supports peak crowd prediction and proactive crowd management for improved campus safety and emergency response. [12][15]

CHAPTER 7

MODULE 4: SECURITY ENGINEERING

7.1 AIM:

To implement a secure and reliable multi-layered security framework for the Campus Crowd Safety System using authentication, encryption, secure APIs, IAM access control, and AWS security services to protect crowd monitoring data, analytics reports, cloud resources, and system operations from unauthorized access and security threats. [5][6][7][8][10][17]

7.2 PROCEDURE FOR SECURITY ENGINEERING

Step 1: Configure secure user authentication for the crowd safety dashboard using session management or token-based authentication for authorized access. [5][6]

Step 2: Implement role-based access control to restrict system access for administrators, event organizers, and authorized security personnel only. [6][10]

Step 3: Secure all API communications using HTTPS and API Gateway security configurations to ensure encryption of data in transit. [5][10]

Step 4: Configure Amazon S3 with secure bucket permissions, encryption settings, and restricted public access policies to protect crowd monitoring datasets. [1][6]

Step 5: Enable encryption for data stored in Amazon S3 to protect crowd logs, analytics reports, alert records, and historical datasets. [1][10]

Step 6: Implement input validation and sanitization in the backend to prevent common web vulnerabilities such as SQL Injection, XSS, and invalid API requests. [5][22]

Step 7: Apply secure headers, CORS controls, and request validation mechanisms within the backend application and API Gateway. [5][21]

Step 8: Configure AWS IAM roles and policies to control access between Lambda, API Gateway, S3, Athena, and CloudWatch using least privilege principles. [6][7][10]

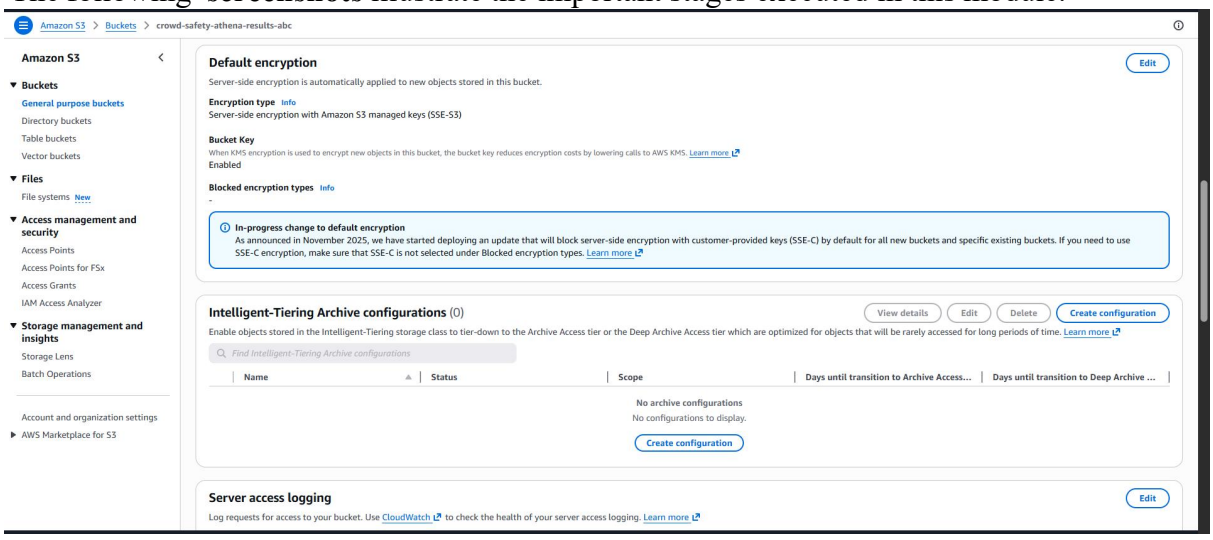
Step 9: Restrict network access using proper cloud security configurations to allow only required inbound and outbound communication between system components. [10][17]

Step 10: Implement secure session handling and automatic timeout mechanisms for dashboard users to prevent unauthorized access. [5][6]

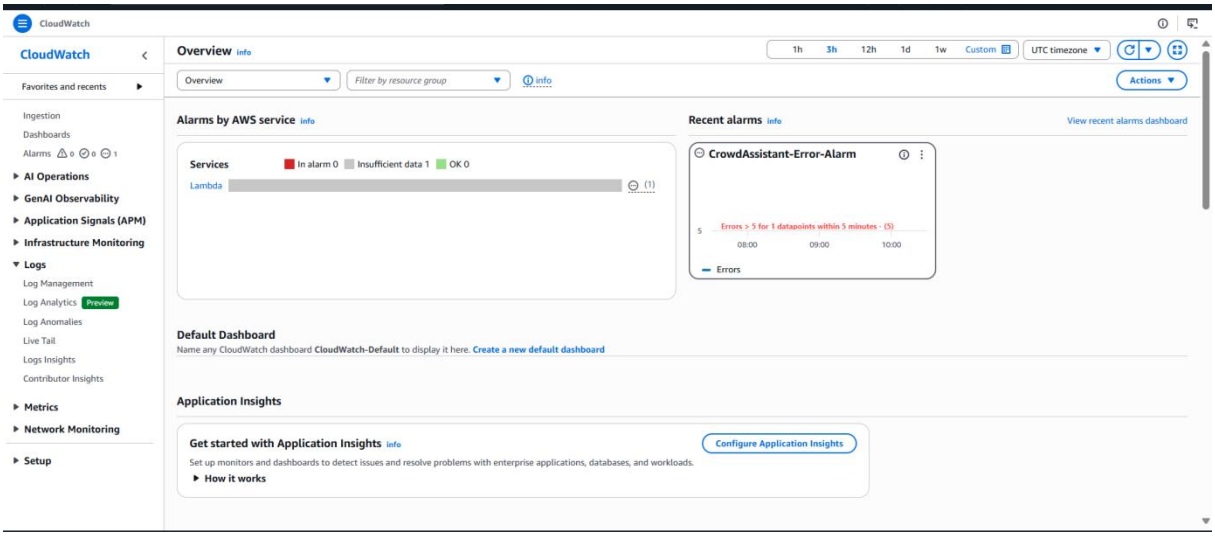
Step 11: Log security-related events and monitor suspicious activities using AWS CloudWatch alarms, logs, and monitoring dashboards. [7][8]

7.3 SCREENSHOTS OF SECURITY ENGINEERING

The following **screenshots** illustrate the important stages executed in this module:



Screenshot 7.1 : Amazon S3 Security Configuration



Screenshot 7.2 : AWS Cloudwatch service

Screenshot 7.3 : Visualization of anomaly detection in gates by crowd live data function

7.1 ACTIVITY DIAGRAM

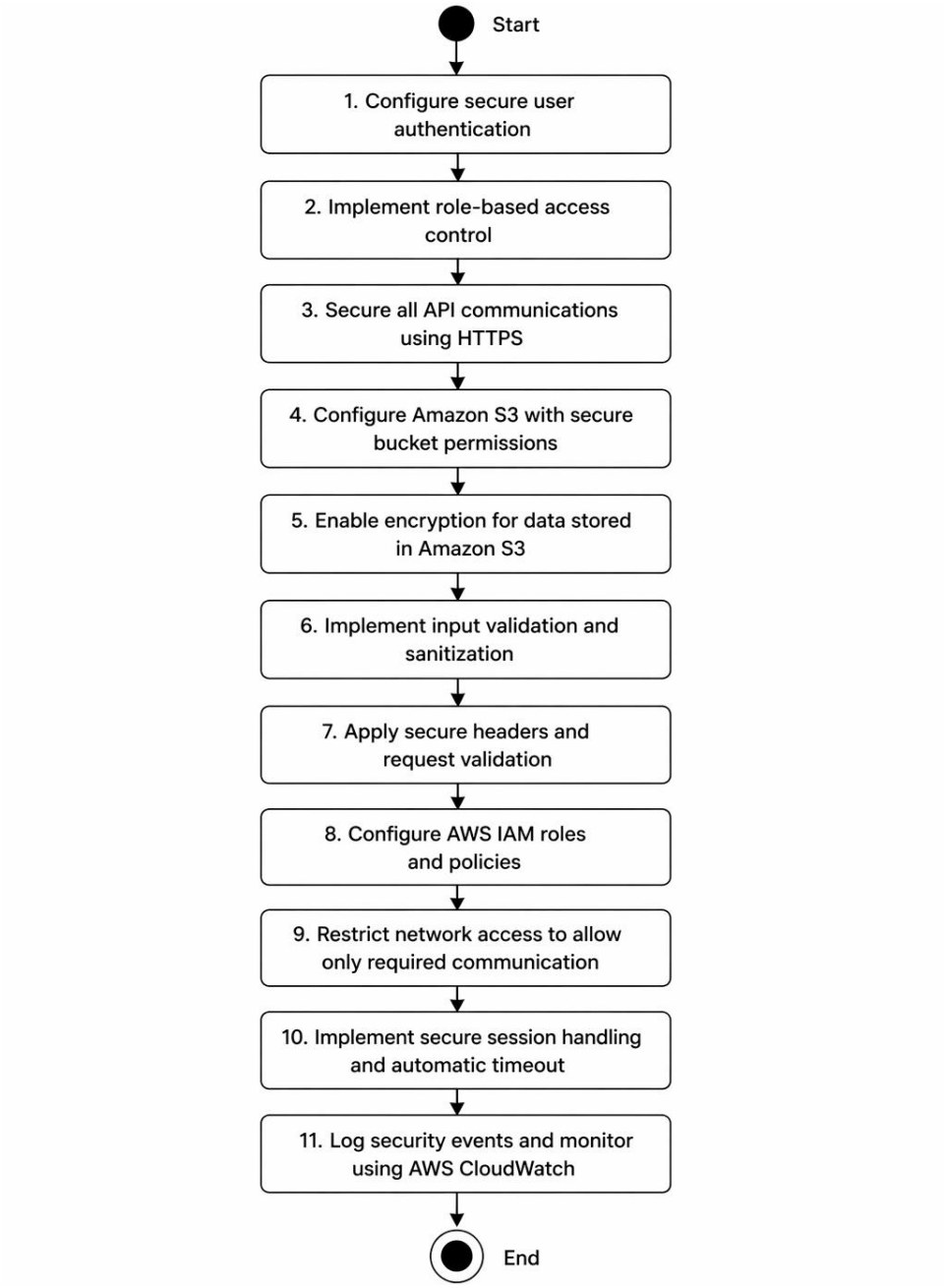


Figure 4.1:Activity Diagram for Security Engineering Module

7.4 RESULT OF SECURITY ENGINEERING

The Security Engineering module was successfully completed. Robust security mechanisms were implemented across the Campus Crowd Safety Monitoring and Anomaly Detection System to ensure secure data handling, protected cloud resources, and reliable network analytics operations.

Key Achievements:

- Secure authentication mechanisms implemented using session management and token-based access control for authorized dashboard users. [5][6]
- Role-based access control enforced to differentiate permissions between administrators, event organizers, and authorized security personnel. [6][10]
- Data encryption applied for secure communication and storage, protecting sensitive information such as crowd logs, analytics reports, and monitoring data. [1][5][10]
- AWS security configurations implemented using Amazon S3, AWS Lambda, IAM roles, and API Gateway with restricted access policies. [1][4][5][6]
- Network-level security ensured through proper configuration of secure API access, HTTPS communication, and controlled access rules. [5][10]
- Protection mechanisms against common web vulnerabilities such as SQL Injection, XSS, CSRF, and invalid requests were successfully implemented. [5][22]
- CloudWatch logging and monitoring enabled to track suspicious activities, audit events, and maintain system reliability. [7][8]
- The security framework now provides a reliable foundation for secure real-time crowd monitoring, intelligent analytics, and protected cloud operations in the Campus Crowd Safety System. [10][17]

CHAPTER 8

CONCLUSION

The development of the **Campus Crowd Safety System** demonstrates an effective approach to monitoring, analyzing, and managing crowd movement in modern campus environments. The project successfully integrates data engineering, cloud computing, artificial intelligence, and security engineering concepts to create a scalable and intelligent crowd safety monitoring system.

By implementing a structured data pipeline, the system ensures efficient collection, storage, and processing of crowd monitoring data. The use of cloud services such as **Amazon S3, AWS Lambda, API Gateway, AWS Glue, and Amazon Athena** enables scalable and reliable system performance. This cloud-based architecture allows the system to handle large volumes of crowd data while maintaining efficiency and flexibility [1][2][3][4][5].

The integration of intelligent anomaly detection techniques helps in identifying unusual crowd patterns such as overcrowded gates, abnormal density spikes, and peak crowd surges. This improves situational awareness and supports proactive decision-making for crowd management and emergency response. Additionally, the AI-powered crowd assistant enhances user interaction by providing meaningful insights into crowd conditions and safety alerts [11][12][15][16][23].

Security measures such as IAM-based access control, data encryption, secure APIs, and monitoring using CloudWatch ensure safe and reliable system operation. These features enhance data protection, system transparency, and secure cloud operations [6][7][10].

In conclusion, the system provides a practical and scalable solution for monitoring campus crowds and improving public safety. It highlights the importance of integrating cloud technologies, intelligent analytics, and security controls to build efficient and reliable safety systems. This project can be further extended with advanced machine learning models, computer vision–based crowd analysis, and real-time predictive analytics to improve accuracy and performance in future implementations.

REFERENCES

[1] Amazon Web Services. (2023). Amazon S3 Documentation — Object Storage, Encryption and Bucket Policies. AWS. https://docs.aws.amazon.com/s3/

[2] Amazon Web Services. (2023). AWS Glue Developer Guide — Crawlers, ETL Jobs and Data Catalog. AWS. https://docs.aws.amazon.com/glue/

[3] Amazon Web Services. (2023). Amazon Athena User Guide — Serverless SQL Queries on S3. AWS. https://docs.aws.amazon.com/athena/

[4] Amazon Web Services. (2023). AWS Lambda Developer Guide — Serverless Functions and Python Runtime. AWS. https://docs.aws.amazon.com/lambda/

[5] Amazon Web Services. (2023). Amazon API Gateway Developer Guide — REST APIs, CORS and Stage Deployment. AWS. https://docs.aws.amazon.com/apigateway/

[6] Amazon Web Services. (2023). AWS IAM User Guide — Roles, LabRole and Least Privilege Policies. AWS. https://docs.aws.amazon.com/iam/

[7] Amazon Web Services. (2023). Amazon CloudWatch User Guide — Dashboards, Alarms and Log Groups. AWS. https://docs.aws.amazon.com/cloudwatch/

[8] Amazon Web Services. (2023). Amazon SNS Developer Guide — Topics and Email Notifications. AWS. https://docs.aws.amazon.com/sns/

[9] Amazon Web Services. (2023). Amazon Bedrock User Guide — Foundation Models and Claude 3 Haiku. AWS. https://docs.aws.amazon.com/bedrock/

[10] Amazon Web Services. (2023). AWS Well-Architected Framework — Serverless Application Lens. AWS. https://docs.aws.amazon.com/wellarchitected/

[11] Helbing, D., & Molnar, P. (1995). Social Force Model for Pedestrian Dynamics. Physical Review E, 51(5), 4282. https://doi.org/10.1103/PhysRevE.51.4282

[12] Still, G. K. (2000). Crowd Dynamics. PhD Thesis, University of Warwick. https://www.crowddynamics.com

[13] Teknomo, K. (2006). Application of Microscopic Pedestrian Simulation Model. Transportation Research Part F, 9(1), 15–27.

[14] Amazon Web Services. (2024). AWS Academy Cloud Foundations. AWS Academy Program — Sri Manakula Vinayagar Engineering College, Dept. of Information Technology, Puducherry. CPPE Module 1.

[15] Amazon Web Services. (2024). AWS Academy Generative AI Foundations. AWS Academy Program — Sri Manakula Vinayagar Engineering College, Dept. of Information Technology, Puducherry. CPPE Module 2.

[16] Amazon Web Services. (2024). AWS Academy Machine Learning Foundations. AWS Academy Program — Sri Manakula Vinayagar Engineering College, Dept. of Information Technology, Puducherry. CPPE Module 3.

[17] Amazon Web Services. (2024). AWS Academy Learner Lab. AWS Academy Program — Sri Manakula Vinayagar Engineering College, Dept. of Information Technology, Puducherry. CPPE Module 4.

[18] Richardson, M., & Domingos, P. (2002). Mining Knowledge-Sharing Sites for Viral Marketing. Proceedings of the 8th ACM SIGKDD, 61–70.

[19] React. (2023). React — A JavaScript Library for Building User Interfaces. Meta Open Source. https://react.dev/

[20] Vite. (2023). Vite — Next Generation Frontend Tooling. https://vitejs.dev/

[21] Axios. (2023). Axios — Promise Based HTTP Client for the Browser and Node.js. https://axios-http.com/

[22] Python Software Foundation. (2023). Python 3.12 Documentation. https://docs.python.org/3.12/

[23] Iglewicz, B., & Hoaglin, D. (1993). How to Detect and Handle Outliers — Z-score Anomaly Detection. ASQC Quality Press, Milwaukee.

[24] Amazon Web Services. (2023). AWS Cloud Adoption Framework — Refactor and Re-architect Strategy. AWS. https://aws.amazon.com/cloud-adoption-framework/